

**Санкт-Петербургский государственный электротехнический
университет «ЛЭТИ» им. В.И. Ульянова (Ленина)**

ClickHouse и информационная безопасность

Лабораторный практикум

Санкт-Петербург
2026

Содержание

1. Теоретическая часть: ClickHouse	4
1.1. Определение	4
1.2. OLAP vs OLTP	4
1.3. Когда нужен ClickHouse	4
1.4. Взаимодействие с ClickHouse	4
1.5. Движки таблиц (Engines)	5
1.5.1. Основные группы движков	5
1.5.2. Семейство MergeTree	5
1.6. Merge — фоновый процесс объединения данных	5
1.7. Партиционирование	5
1.7.1. Типичные ошибки партиционирования	6
1.8. TTL — управление жизненным циклом данных	6
1.9. ORDER BY — физический порядок хранения	6
1.9.1. Проектирование ORDER BY	6
1.10. PRIMARY KEY	7
1.11. Проблема дублей и ReplacingMergeTree	7
1.11.1. Способы получить дедуплицированные данные	7
1.12. Распределённая архитектура	7
1.12.1. ZooKeeper и ClickHouse Keeper	8
1.12.2. Шарды и реплики	8
1.12.3. Локальные и распределённые таблицы	8
2. Введение в лабораторный практикум	9
3. Лабораторная работа 1: RBAC в ClickHouse	10
3.1. Цель	10
3.2. Подготовка	10
3.3. Часть 1: Роли и привилегии	10
3.3.1. Задание	10
3.3.2. Контрольные вопросы	11
3.4. Часть 2: Row Policies	11
3.4.1. Задание	11
3.4.2. Контрольные вопросы	11
3.5. Часть 3: Квоты и ограничения (дополнительно)	11
3.5.1. Задание	11
3.5.2. Контрольные вопросы	11
3.6. Часть 4: Проверка модели доступа	11
3.6.1. Задание	11
3.6.2. Вывод	12
4. Лабораторная работа 2: Обезличивание персональных данных	13
4.1. Цель	13
4.2. Исходные данные	13
4.3. Часть 1: Наивный подход (ловушка)	13
4.3.1. Задание	13
4.4. Часть 2: Попытка через ReplacingMergeTree	13
4.4.1. Задание	13
4.5. Часть 3: Правильный подход — ETL-пайплайн	13
4.5.1. Задание	13
4.5.2. Вывод	14
5. Лабораторная работа 3: Аудит логов безопасности	15
5.1. Цель	15
5.2. Задание	15
5.2.1. Часть 1: Схема данных	15

5.2.2. Часть 2: Генерация событий	15
5.2.3. Часть 3: Аналитические запросы	15
5.2.4. Часть 4: Materialized Views для агрегации	15
5.2.5. Связь с архитектурой ClickHouse	15
6. Лабораторная работа 4: Private Set Intersection	16
6.1. Цель	16
6.2. Задание	16
6.2.1. Часть 1: Подготовка данных	16
6.2.2. Часть 2: Реализация PSI	16
6.2.3. Часть 3: Верификация	16
6.2.4. Связь с архитектурой ClickHouse	16

1. Теоретическая часть: ClickHouse

1.1. Определение

ClickHouse — это распределённая СУБД¹ с открытым исходным кодом, созданная для аналитической обработки запросов (OLAP²) и хранения больших объёмов данных. Изначально разработана в Яндексе для Яндекс.Метрики — системы веб-аналитики, обрабатывающей миллиарды событий в сутки.

ClickHouse оптимизирован для сложных аналитических запросов, дашбордов и отчётности с низкой латентностью. Это позволяет аналитикам и инженерам строить интерактивные витрины и системы мониторинга поверх петабайтных объёмов данных, сохраняя высокую производительность даже при большом числе конкурентных запросов.

1.2. OLAP vs OLTP

	OLTP	OLAP
Назначение	Быстрые короткие транзакции: вставка, обновление, чтение отдельных записей	Сложные аналитические запросы: агрегация, группировка, анализ больших объёмов исторических данных
Оптимизация	Минимальная задержка, высокая согласованность при большом числе одновременных пользователей	Максимальная пропускная способность, высокая скорость сканирования миллионов/миллиардов записей
Схемы	Нормализованные, строгие ограничения, строковое хранение	Денормализованные (звезда, снежинка), колоночное хранение
Примеры	Платежи, заказы, CRM ³ , банковские транзакции	BI ⁴ , дашборды, Ad-hoc анализ ⁵ запросы, мониторинг

Table 1: Сравнение OLTP и OLAP

1.3. Когда нужен ClickHouse

Подходит:

- Аналитика на больших объёмах данных (шардирование, горизонтальное масштабирование).
- Высокая скорость агрегирования.
- Интерактивные дашборды и мониторинг.
- Поточковая обработка — анализ данных в реальном времени.

Не подходит:

- Нужны полноценные транзакции (многотабличные ACID-транзакции, сложные BEGIN/COMMIT/ROLLBACK сценарии). Экспериментальная поддержка транзакций существует, но ограничена.
- Маленькие датасеты (для этого есть DuckDB).

1.4. Взаимодействие с ClickHouse

ClickHouse поддерживает несколько протоколов доступа:

¹Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

²Online Analytical Processing — класс систем для аналитической обработки данных. Ориентирован на выполнение сложных агрегирующих запросов по большим объёмам данных, использует колоночное хранение для оптимизации производительности аналитики.

³Customer Relationship Management — система управления взаимоотношениями с клиентами. Программное обеспечение для автоматизации стратегий взаимодействия с клиентами, включая продажи, маркетинг, техподдержку и аналитику клиентских данных.

⁴Business Intelligence — технологии и практики сбора, интеграции, анализа и представления бизнес-информации. Включает инструменты для создания отчётов, дашбордов, визуализации данных и поддержки принятия управленческих решений на основе данных.

⁵Разовые, незапланированные аналитические запросы для исследования данных или проверки гипотез. В отличие от регулярных отчётов, выполняются по требованию без предварительной настройки ETL-пайплайнов. Требуется интерактивной производительности и гибкого SQL.

- **Нативный TCP-протокол** — максимальная производительность, используется clickhouse-client и большинством драйверов.
- **HTTP / HTTPS интерфейс** — удобен для интеграции с веб-приложениями и мониторингом.
- **MySQL / PostgreSQL wire protocol** — совместимость с существующими клиентами и инструментами.
- **JDBC / ODBC драйверы** — для Java-приложений и BI-инструментов.

Вне зависимости от выбранного протокола, общение происходит на обычном SQL:

```
SELECT t1.id, t2.name
FROM t1
JOIN t2 ON t1.id = t2.id
```

1.5. Движки таблиц (Engines)

Поведение таблицы полностью определяется выбранным движком (engine). Именно engine определяет:

- как данные физически хранятся на диске,
- можно ли выполнять merge,
- поддерживается ли репликация,
- как работает дедупликация,
- возможны ли TTL и мутации (UPDATE/DELETE).

Выбор engine — это архитектурное решение. Его нельзя легко изменить без полной пересборки таблицы.

1.5.1. Основные группы движков

MergeTree-семейство — используется в 90% продакшн-кейсов. Предназначено для больших объёмов аналитических данных.

Log / TinyLog — простые движки без merge и индексов. Используются для временных таблиц и тестирования.

Distributed — логический движок для работы с шардами. Сам данные не хранит.

Special engines — Kafka, Buffer, Dictionary — используются для интеграций и потоковой обработки.

1.5.2. Семейство MergeTree

- **MergeTree** — базовый вариант.
- **ReplicatedMergeTree** — с репликацией между нодами.
- **ReplacingMergeTree** — дедупликация по ключу при merge.
- **SummingMergeTree** — агрегация числовых колонок при merge.
- **AggregatingMergeTree** — хранение агрегатных состояний.
- **Collapsing / VersionedCollapsingMergeTree** — хранение событий с отменами.

1.6. Merge — фоновый процесс объединения данных

INSERT в ClickHouse никогда не переписывает существующие данные — он создаёт новые части (parts).

Merge — это фоновый процесс, который объединяет эти части.

Это не оптимизация, а фундаментальная часть архитектуры. Система предполагает, что:

- данные пишутся быстро (append-only),
- порядок и консистентность достигаются позже (при merge),
- аналитика допускает небольшую задержку в «идеальности» данных.

1.7. Партиционирование

Партиции — это логическое разбиение таблицы на независимые части (обычно по дате), задаваемое выражением PARTITION BY.

Партиции позволяют:

- быстро удалять данные (DROP PARTITION — мгновенная операция),
- ограничивать scope merge,
- эффективно применять TTL,
- ускорять запросы с фильтрами по ключу партиционирования.

1.7.1. Типичные ошибки партиционирования

Основная ошибка — слишком мелкие партиции (например, партиционирование по `user_id` или `uuid`).

Количество партиций	Оценка
Десятки – сотни	Норма
1–2 тысячи	Допустимо в редких случаях
Десятки тысяч	Плохо
Сотни тысяч и более	Катастрофа

Table 2: Рекомендации по количеству партиций на таблицу

1.8. TTL — управление жизненным циклом данных

TTL — механизм автоматического удаления или перемещения данных по времени без внешних задач. Работает только в MergeTree-движках.

Важно: TTL не удаляет данные сразу, а дожидается merge. Данные будут существовать дольше указанного срока. При необходимости можно вручную запустить `OPTIMIZE TABLE ... FINAL`.

TTL работает на уровне частей (parts), а не партиций. Если TTL совпадает с границей партиций и включена настройка `ttl_only_drop_parts = 1`, ClickHouse целиком удаляет части, в которых все строки истекли — это быстрая операция. Без этой настройки (по умолчанию выключена) удаляются отдельные строки через мутации, что значительно дороже.

```
CREATE TABLE events_optimized (  
    event_date Date,  
    user_id UInt64,  
    event_type String,  
    data String  
) ENGINE = MergeTree()  
PARTITION BY toYYYYMM(event_date) -- Партиции по месяцам  
ORDER BY (event_date, user_id)  
TTL event_date + INTERVAL 3 MONTH; -- TTL кратен партициям
```

1.9. ORDER BY — физический порядок хранения

`ORDER BY` в определении таблицы определяет физический порядок хранения данных на диске. Это **не** порядок вывода в `SELECT` — для этого нужен отдельный `ORDER BY` в запросе.

Если поле из `WHERE` совпадает с префиксом `ORDER BY`, ClickHouse:

- читает минимальное количество данных,
- эффективно использует min/max индексы (sparse index),
- существенно уменьшает I/O.

1.9.1. Проектирование ORDER BY

`ORDER BY` должен:

- соответствовать самым частым фильтрам,
- **начинаться с колонок низкой кардинальности** (ascending cardinality),
- учитывать типичные паттерны запросов.

Это противоположно интуиции из OLTP (где B-tree индексы выигрывают от высокой селективности первой колонки). В ClickHouse sparse index работает иначе: первая колонка использует бинарный поиск, а для последующих колонок применяется generic exclusion search, который эффективен только когда предыдущая колонка имеет мало уникальных значений (одно значение покрывает много гранул). Кроме того, низкая кардинальность ведущей колонки значительно улучшает сжатие.

```
-- Хорошо: event_type (5-10 значений) первым, затем user_id (миллионы)  
ORDER BY (event_type, user_id, event_date);
```

```
-- Плохо: user_id первым — высокая кардинальность ломает
```

```
-- эффективность sparse index для последующих колонок
ORDER BY (user_id, event_type, event_date);
```

1.10. PRIMARY KEY

PRIMARY KEY определяет разреженный индекс и **ничего больше**.

Ключевые отличия от OLTP-систем:

- **ПК не гарантирует уникальность** — дубли разрешены.
- ПК обязан быть **префиксом** ORDER BY.
- Если PRIMARY KEY не указан — автоматически равен ORDER BY.

```
-- PRIMARY KEY автоматически = (timestamp, server_id, level)
ORDER BY (timestamp, server_id, level);
```

```
-- Можно указать более короткий PK для экономии памяти:
ORDER BY (timestamp, server_id, level, request_id)
PRIMARY KEY (timestamp, server_id);
-- level и request_id участвуют в сортировке, но не в индексе
```

1.11. Проблема дублей и ReplacingMergeTree

Поскольку PRIMARY KEY не гарантирует уникальность, а UPSERT отсутствует, для управления дублями используется ReplacingMergeTree.

```
CREATE TABLE users_replacing (
    user_id UInt64,
    name String,
    email String,
    updated_at DateTime -- Колонка версии
) ENGINE = ReplacingMergeTree(updated_at)
PRIMARY KEY user_id;
```

При вставке строки с тем же ключом дубли **сохраняются**:

```
INSERT INTO users_replacing VALUES
(1, 'Alice', 'alice@example.com', '2026-01-01 10:00:00');
INSERT INTO users_replacing VALUES
(1, 'Alice Updated', 'new@example.com', '2026-01-01 11:00:00');
```

```
-- Дубли видны!
SELECT * FROM users_replacing WHERE user_id = 1;
-- → 2 строки
```

1.11.1. Способы получить дедуплицированные данные

1. **FINAL** — дедупликация «на лету» при чтении:

```
SELECT * FROM users_replacing FINAL;
```

2. **argMax** — ручная дедупликация через агрегацию:

```
SELECT
    user_id,
    argMax(name, updated_at) AS name,
    argMax(email, updated_at) AS email,
    max(updated_at) AS updated_at
FROM users_replacing
GROUP BY user_id;
```

3. **OPTIMIZE TABLE ... FINAL** — принудительный merge:

```
OPTIMIZE TABLE users_replacing FINAL;
-- После этого дубли физически удалены
```

4. **Materialized View** — дедупликация при вставке в отдельную таблицу.

1.12. Распределённая архитектура

1.12.1. ZooKeeper и ClickHouse Keeper

ZooKeeper используется как координационная система в распределённых кластерах ClickHouse: репликация, распределённые DDL, синхронизация состояния. ZooKeeper — отдельный сервис на Java.

ClickHouse Keeper — более новая альтернатива, написанная на C++ и оптимизированная под ClickHouse.

1.12.2. Шарды и реплики

ClickHouse для распределения данных использует два ключевых понятия:

Шард (shard) — разделение данных на части. Каждый шард хранит свой фрагмент таблицы. Например, таблица логов за 12 месяцев:

- Шард 1: январь–апрель
- Шард 2: май–август
- Шард 3: сентябрь–декабрь

Запись попадает в нужный шард, а чтение опрашивает только те шарды, где находятся нужные данные.

Реплика (replica) — копия шарда для отказоустойчивости. Каждый шард может иметь несколько реплик с одинаковыми данными:

	Реплика 1	Реплика 2
Шард 1	Сервер А	Сервер В
Шард 2	Сервер С	Сервер D
Шард 3	Сервер Е	Сервер F

Table 3: Пример распределения: 3 шарда × 2 реплики

Итого:

- **Шарды** — горизонтальное масштабирование (больше данных).
- **Реплики** — отказоустойчивость и балансировка чтения.

1.12.3. Локальные и распределённые таблицы

Нераспределённая таблица (обычный MergeTree):

```
CREATE TABLE events_local (  
    id UInt64,  
    ts DateTime,  
    user_id UInt32  
) ENGINE = MergeTree()  
ORDER BY (id);
```

Распределённая таблица с репликацией:

```
CREATE TABLE events_local (  
    id UInt64,  
    ts DateTime,  
    user_id UInt32  
) ENGINE = ReplicatedMergeTree(  
    '/clickhouse/tables/{shard}/events_local',  
    '{replica}'  
)  
ORDER BY (id)  
ON CLUSTER my_cluster;
```


2. Введение в лабораторный практикум

Практикум состоит из четырёх лабораторных работ на стыке аналитических баз данных и информационной безопасности. Все лабы используют ClickHouse — теоретические основы рассмотрены в предыдущем разделе.

Ключевая идея, которая проходит через все лабы: ClickHouse — это колоночная аналитическая СУБД, а не OLTP-система. Каждая лаба подсвечивает конкретное следствие этой архитектуры для задач ИБ:

- **Лаба 1 (RBAC):** роли, Row Policies, квоты; мутации тяжёлые → разграничение прав на ALTER критически важно.
- **Лаба 2 (Обезличивание):** нельзя просто UPDATE ПДн → нужен ETL-пайплайн; студент сталкивается с ReplacingMergeTree и понимает, что это не замена UPSERT.
- **Лаба 3 (Аудит):** append-only природа — **фича** для логов безопасности, удалить следы сложно.
- **Лаба 4 (PSI):** иммутабельность данных усиливает изоляцию между участниками протокола.

3. Лабораторная работа 1: RBAC в ClickHouse

3.1. Цель

Развернуть ClickHouse (Docker), создать таблицу с тестовыми данными, настроить ролевую модель доступа: пользователи, роли, привилегии, Row Policies, квоты. Убедиться, что разграничение прав в OLAP-системе работает иначе, чем в OLTP.

3.2. Подготовка

1. Развернуть ClickHouse через Docker Compose.
2. Подключиться к серверу через clickhouse-client.
3. Создать таблицу events и наполнить тестовыми данными:

```
CREATE TABLE events (  
    event_id UInt32,  
    timestamp DateTime,  
    user_id UInt32,  
    department String,  
    event_type String,  
    status String  
) ENGINE = MergeTree()  
PARTITION BY toYYYYMM(timestamp)  
ORDER BY (department, event_type, timestamp);  
  
INSERT INTO events  
SELECT  
    number,  
    now() - randUniform(0, 90*24*3600),  
    rand() % 1000,  
    ['sales', 'engineering', 'support'][rand() % 3 + 1],  
    ['login', 'logout', 'access', 'modify', 'delete']  
        [rand() % 5 + 1],  
    ['ok', 'error', 'denied'][rand() % 3 + 1]  
FROM numbers(50000);
```

3.3. Часть 1: Роли и привилегии

3.3.1. Задание

1. Создать три роли с разным уровнем доступа:

```
CREATE ROLE analyst;  
CREATE ROLE ingester;  
CREATE ROLE admin;  
  
-- analyst: только чтение  
GRANT SELECT ON default.events TO analyst;  
  
-- ingester: только вставка  
GRANT INSERT ON default.events TO ingester;  
  
-- admin: полный доступ (включая ALTER = мутации)  
GRANT ALL ON default.* TO admin;
```

2. Создать пользователей и назначить роли:

```
CREATE USER viewer IDENTIFIED BY 'viewer123'  
    DEFAULT ROLE analyst;  
CREATE USER loader IDENTIFIED BY 'loader123'  
    DEFAULT ROLE ingester;  
CREATE USER superuser IDENTIFIED BY 'admin123'  
    DEFAULT ROLE admin;
```

3. Проверить привилегии — подключиться как viewer:

```
clickhouse-client --user viewer --password viewer123
```

- `SELECT count() FROM events` — работает.
- `INSERT INTO events VALUES (...)` — ошибка `Not enough privileges`.
- `ALTER TABLE events DELETE WHERE 1` — ошибка `Not enough privileges`.

3.3.2. Контрольные вопросы

- Чем `GRANT ALL` отличается от перечисления `GRANT SELECT, INSERT, ALTER`?
- Что произойдёт, если пользователю назначены две роли с конфликтующими правами?

3.4. Часть 2: Row Policies

3.4.1. Задание

1. Создать Row Policy — `analyst` видит только данные своего подразделения:

```
CREATE ROW POLICY sales_only ON events
  FOR SELECT USING department = 'sales'
  TO analyst;
```

2. Подключиться как `viewer` и выполнить:

```
SELECT department, count() FROM events GROUP BY department;
```

Результат: только строки `sales`. Остальные подразделения не видны.

3. Добавить вторую политику для другого пользователя:

```
CREATE USER eng_viewer IDENTIFIED BY 'eng123'
  DEFAULT ROLE analyst;
```

```
CREATE ROW POLICY eng_only ON events
  FOR SELECT USING department = 'engineering'
  TO eng_viewer;
```

4. Убедиться, что `eng_viewer` видит только `engineering`, а `viewer` — только `sales`.

3.4.2. Контрольные вопросы

- Чем Row Policy в ClickHouse отличается от VIEW с фильтром?
- Можно ли обойти Row Policy через подзапрос или JOIN?

3.5. Часть 3: Квоты и ограничения (дополнительно)

3.5.1. Задание

1. Настроить квоту для роли `analyst`:

```
CREATE QUOTA analyst_quota
  FOR INTERVAL 1 MINUTE MAX QUERIES 10,
  FOR INTERVAL 1 HOUR MAX RESULT_ROWS 100000
  TO analyst;
```

2. Подключиться как `viewer` и выполнить 11 запросов подряд — на 11-м получить ошибку `Quota exceeded`.
3. Настроить ограничение на уровне профиля:

```
CREATE SETTINGS PROFILE analyst_profile
  SETTINGS max_execution_time = 5,
           max_memory_usage = 100000000 -- 100 MB
  TO analyst;
```

4. Выполнить тяжёлый запрос от имени `viewer` и убедиться, что он прерывается по таймауту.

3.5.2. Контрольные вопросы

- Зачем нужны квоты в аналитической системе, если запросы только на чтение?
- Чем квоты отличаются от настроек профиля (`SETTINGS PROFILE`)?

3.6. Часть 4: Проверка модели доступа

3.6.1. Задание

1. Заполнить таблицу, подключаясь от имени каждого пользователя и проверяя каждую операцию:

Операция	viewer	loader	superuser
SELECT * FROM events	?	?	?
SELECT (видит все department?)	?	?	?
INSERT INTO events VALUES (...)	?	?	?
DROP TABLE events	?	?	?
11-й запрос за минуту (квота)	?	—	—

Table 4: Матрица доступа

1. Убедиться, что `system.query_log` фиксирует все действия всех пользователей, включая отклонённые запросы:

```
SELECT user, query, type, exception
FROM system.query_log
WHERE event_time > now() - INTERVAL 10 MINUTE
ORDER BY event_time DESC
LIMIT 20;
```

3.6.2. Вывод

RBAC в ClickHouse позволяет гранулярно разграничить доступ: от ролей и привилегий до фильтрации строк (Row Policies) и ограничения нагрузки (квоты). Все действия пользователей — включая отказы — аудируемы через `system.query_log`.

4. Лабораторная работа 2: Обезличивание персональных данных

4.1. Цель

Реализовать пайплайн анонимизации ПДн, по пути столкнувшись с тем, что привычный подход «просто обновить поля» в ClickHouse не работает.

4.2. Исходные данные

Синтетический датасет `raw_users` (~10K строк): ФИО, email, телефон, дата рождения, город, сумма транзакций.

4.3. Часть 1: Наивный подход (ловушка)

4.3.1. Задание

1. Попробовать обезличить данные «на месте»:

```
ALTER TABLE raw_users
  UPDATE email = SHA256(email),
         phone = concat('***', substring(phone, -4))
 WHERE 1
```

2. Наблюдать за мутацией: `SELECT * FROM system.mutations WHERE is_done = 0`.
3. Замерить время выполнения.
4. Выполнить `SELECT` во время мутации — увидеть, что часть строк уже обезличена, а часть — ещё нет (частичная видимость).
5. Зафиксировать проблемы:
 - Мутация медленная и ресурсоёмкая.
 - Во время выполнения данные в неконсистентном состоянии.
 - Откатить мутацию нельзя.

4.4. Часть 2: Попытка через `ReplacingMergeTree`

4.4.1. Задание

1. Создать таблицу `users_replacing` на движке `ReplacingMergeTree(version)` с тем же ключом, что и `raw_users`.
2. Написать Python-скрипт, который:
 - Читает данные из `raw_users`.
 - Применяет анонимизацию (хеширование email, маскирование телефона).
 - Вставляет обезличенные строки в `users_replacing` с `version = 2`.
3. Выполнить `SELECT ... FROM users_replacing` — увидеть дубли (оригиналы с `version=1` и обезличенные с `version=2`).
4. Выполнить `SELECT ... FROM users_replacing FINAL` — увидеть только обезличенные строки.
5. Выполнить `OPTIMIZE TABLE users_replacing FINAL` — дожидаться физического слияния.
6. Зафиксировать проблемы:
 - `FINAL` замедляет запросы.
 - `OPTIMIZE ... FINAL` — тяжёлая операция, по сути аналог мутации.
 - Оригинальные данные физически остаются на диске до слияния — с точки зрения ИБ это плохо.

4.5. Часть 3: Правильный подход — ETL-пайплайн

4.5.1. Задание

1. Написать пайплайн на Python:
 - Чтение из `raw_users` (можно батчами через `LIMIT ... OFFSET` или `clickhouse-connect`).
 - Трансформация: маскирование ФИО, хеширование email (SHA-256 с солью), генерализация города (город → регион), бакетирование возраста (25 → «20–29»).
 - Запись в новую таблицу `anonymized_users` (обычный `MergeTree`).
2. Сравнить аналитику до и после анонимизации:
 - Распределение по регионам — должно сохраниться.

- Средняя сумма транзакций по возрастным группам — должна быть близка.
- Точный поиск по email — должен быть невозможен без знания соли.

4.5.2. Вывод

В OLAP-системах данные не «правят на месте», а перекладывают трансформированными. Это не ограничение, а архитектурный паттерн: исходные данные иммутабельны, каждая трансформация создаёт новый слой. Для ИБ это означает, что обезличивание — это ETL-задача, а не UPDATE.

5. Лабораторная работа 3: Аудит логов безопасности

5.1. Цель

Построить мини-SIEM на базе ClickHouse: сбор, хранение и анализ security-событий. Append-only природа ClickHouse здесь работает **в пользу** безопасности.

5.2. Задание

5.2.1. Часть 1: Схема данных

1. Спроектировать таблицу `security_events` на движке MergeTree с партиционированием по дате (`toYYYYMM(timestamp)`).
2. Поля: `timestamp`, `event_type`, `source_ip`, `user_id`, `action`, `status`, `details`.
3. Выбрать ключ сортировки, оптимальный для типичных аналитических запросов (поиск по времени, по IP, по типу события).

5.2.2. Часть 2: Генерация событий

1. Написать Python-генератор, который создаёт поток security-событий:
 - Логины (успешные/неуспешные), в том числе серии brute-force.
 - Доступ к ресурсам (разрешённый/запрещённый).
 - Изменения привилегий.
2. Загрузить ~100K событий за «3 месяца».

5.2.3. Часть 3: Аналитические запросы

1. Обнаружение brute-force: найти IP-адреса с более чем N неудачных попыток логина за 5 минут.
2. Аномальная активность: пользователи с нетипично большим количеством действий ($> 3\sigma$ от среднего).
3. Эскалация привилегий: цепочки событий «логин → изменение роли → доступ к ресурсу» за короткий промежуток.

5.2.4. Часть 4: Materialized Views для агрегации

1. Создать Materialized View `mv_failed_logins_per_minute`, который автоматически агрегирует неудачные логины по минутам и IP-адресам.
2. Сравнить производительность запроса к сырым данным и к Materialized View.

5.2.5. Связь с архитектурой ClickHouse

Append-only природа MergeTree означает, что злоумышленник не может легко удалить следы из логов: DELETE — это мутация, которая оставляет записи в `system.mutations` и `system.query_log`. Materialized View — это не кэш, а отдельная таблица, куда данные попадают при вставке; удаление из исходной таблицы не удаляет агрегаты.

6. Лабораторная работа 4: Private Set Intersection

6.1. Цель

Реализовать упрощённый протокол Private Set Intersection (PSI) на основе Diffie-Hellman: две «организации» находят общих клиентов, не раскрывая свои полные базы.

6.2. Задание

6.2.1. Часть 1: Подготовка данных

1. Создать две таблицы в ClickHouse: `org_a_clients` и `org_b_clients` с частичным пересечением по идентификатору (email).
2. Каждая таблица — обычный MergeTree, ~100K записей.

6.2.2. Часть 2: Реализация PSI

1. Реализовать на Python протокол PSI на эллиптических кривых (или упрощённый DH):
 - Организация А хеширует свои email на кривую и отправляет точки организации В.
 - Организация В делает то же и вычисляет общий секрет.
 - Пересечение определяется по совпадению результатов.
2. Оркестрация: Python читает данные из ClickHouse, выполняет криптографические операции, записывает результат обратно.

6.2.3. Часть 3: Верификация

1. Проверить корректность: результат PSI должен совпадать с INNER JOIN по открытым данным.
2. Измерить overhead протокола по сравнению с прямым JOIN.

6.2.4. Связь с архитектурой ClickHouse

Данные в таблицах иммутабельны: организация В не может модифицировать данные организации А, даже имея доступ к ClickHouse. Row Policies (из Лабы 1) дополнительно изолируют доступ. Результаты PSI записываются в отдельную таблицу — append-only, с полной аудируемостью (из Лабы 3).